

Práctica Final – Documento Técnico

Caso: Coquito Amarillo S.A.S.

Johan Sebastian Barrera Rua
Miguel Ángel González Estrada

Ingeniería de Software
Fundación Universitaria María Cano

Construcción de Software I

Medellín, 29 de mayo de 2026

Tabla contenido

1. Introducción	4
2. Descripción del Problema	4
2.1. Problemática Identificada	4
2.2. Causas Raíz	5
3. Objetivos	5
3.1. Objetivo General.....	5
3.2. Objetivos Específicos	5
4. Planteamiento de la Solución	5
4.1. Módulo de Autenticación	6
4.2. Módulo de Catálogo	6
4.3. Módulo de Carrito de Compras	6
4.4. Módulo de Pedidos	6
4.5. Módulo de Pagos	6
4.6. Módulo de Notificaciones	6
5. Actores del Sistema.....	6
6. Arquitectura del sistema.....	7
6.1. Módulos del Sistema	7
7. Requerimientos del sistema.....	8
7.1. Requerimientos Funcionales.....	8
7.2. Requerimientos No Funcionales.....	10
8. Casos de Uso	10
8.1. CU-01: Registrarse e iniciar sesión	11
8.2. CU-02: Explorar catálogo de productos	12
8.3. CU-03: Gestionar carrito de compras	12
8.4. CU-04: Registrar pedido.....	13
8.5. CU-05: Procesar pago.....	13
8.6. CU-06: Cancelar pedido	14
8.7. CU-07: Consultar historial de pedidos	14
8.8. CU-08: Gestionar inventario de productos	15
8.9. CU-09: Gestionar pedidos del sistema	15
8.10. CU-10: Actualizar estado logístico del pedido	16
9. Clases del Sistema.....	17
9.1. Relaciones Entre Clases.....	17

10.	Diagramas de Clase	18
11.	Organización del Equipo	18
11.1.	Integrantes y Roles.....	18
11.2.	Metodología de Trabajo	19
12.	Normas y Procedimientos de Calidad	19
12.1.	ISO/IEC 25010 — Modelo de Calidad del Producto	19
12.2.	IEEE 730 — Plan de Aseguramiento de Calidad	19
13.	Actividades de Aseguramiento de Calidad.....	19
13.1.	Estrategia de Pruebas	19
13.2.	Criterios de Aceptación por Módulo	20
14.	Herramientas y Tecnologías	20
15.	Métricas y Criterios de Aceptación	21
15.1.	Métricas de Calidad del Código.....	21
15.2.	Criterios de Aceptación del Producto Final	21
16.	Implementación del Sistema.....	21
16.1.	Backend y API REST	21
16.2.	Base de Datos.....	21
16.3.	Gestión de Pedidos.....	21
16.4.	Gestión de Pagos.....	22
16.5.	Validaciones y Seguridad	22
16.6.	Pruebas y CI/CD	22
17.	Gestión de Cambios.....	22
17.1.	Procedimiento de Control de Cambios	22
17.2.	Clasificación de Cambios	22
18.	Operación, Pruebas y Mejoras.....	22
19.	Revisión y Aprobación	23
19.1.	Cronograma de Sprints	23
19.2.	Firmas de Aprobación.....	23
20.	Reflexión	23
21.	Conclusiones.....	23

1. Introducción

Actualmente, las organizaciones dedicadas al comercio electrónico requieren soluciones tecnológicas que permitan gestionar de manera eficiente sus procesos de ventas, pedidos, pagos e inventario. La digitalización de los procesos comerciales ha incrementado las expectativas de los clientes respecto a la velocidad, confiabilidad y disponibilidad de los servicios ofrecidos por las plataformas de compra en línea.

La ausencia de una solución integrada genera dificultades relacionadas con la trazabilidad de los pedidos, inconsistencias en el manejo del stock, retrasos en la confirmación de pagos y limitaciones en la experiencia del cliente. Asimismo, la empresa carece de mecanismos automatizados para validar operaciones críticas, controlar estados de pedidos y garantizar la integridad de la información procesada.

El presente documento describe el proceso de concepción, diseño e implementación de una WebApp con API-REST que permita gestionar clientes, productos, carrito de compras, pedidos y pagos, garantizando calidad, escalabilidad y una experiencia de usuario fluida.

2. Descripción del Problema

Coquito Amarillo S.A.S. es una empresa colombiana dedicada a la comercialización de ropa, calzado y accesorios deportivos, con presencia en las ciudades de Medellín, Bogotá y Cali. Su público objetivo está conformado por personas naturales interesadas en productos deportivos, a quienes atiende principalmente a través de su plataforma de comercio electrónico.

La plataforma digital actual presenta múltiples fallas técnicas que han generado una caída del 35% en las ventas en línea durante el último trimestre. La gerencia ha tomado la decisión de reemplazar completamente la plataforma por una solución desarrollada internamente, bajo los estándares de calidad exigidos por las normas ISO/IEC 25010 e IEEE 730.

En este contexto, la organización presenta limitaciones en la gestión de sus procesos operativos y comerciales, ya que no cuenta con un sistema centralizado que permita administrar de forma eficiente el catálogo de productos, el carrito de compras, los pedidos, el control de inventario y el procesamiento de pagos.

Ante esta situación, surge la necesidad de desarrollar una solución de software que permita optimizar los procesos comerciales de la organización mediante una arquitectura modular, escalable y orientada a servicios.

2.1. Problemática Identificada

El diagnóstico técnico del sistema actual de Coquito Amarillo S.A.S. revela las siguientes deficiencias críticas que afectan directamente el desempeño comercial y la experiencia de los clientes: Los tiempos de carga de la plataforma superan los 8 segundos, lo que genera una alta tasa de abandono antes de completar la compra. El carrito de compras no persiste entre recargas de página ni entre sesiones, y no permite modificar la cantidad de productos ya agregados.

El botón de pago presenta fallos sin retroalimentación al usuario, no se valida la disponibilidad de stock antes de agregar productos al carrito, y no se envía confirmación de compra por correo electrónico al finalizar la transacción. Como consecuencia, la tasa de abandono de carrito supera el 70% y las ventas en línea han caído un 35% en el último trimestre.

2.2. Causas Raíz

El análisis técnico identifica las siguientes causas de fondo: el carrito carece de persistencia en base de datos y de controles de edición adecuados; la integración con la pasarela de pago no gestiona correctamente respuestas de error ni estados de carga en la interfaz; el sistema no realiza consultas al inventario en tiempo real al agregar productos ni al mostrar el catálogo; no existe un módulo integrado de notificaciones externas; y la lógica de recálculo del carrito no está vinculada a los eventos de cambio de cantidad en la interfaz.

3. Objetivos

3.1. Objetivo General

Desarrollar un sistema de gestión de ventas y pedidos mediante una arquitectura basada en servicios REST, que permita administrar productos, clientes, pagos, pedidos e inventario de forma segura y eficiente.

3.2. Objetivos Específicos

- Implementar un módulo de autenticación que valide credenciales y controle el acceso mediante bloqueo automático ante intentos fallidos.
- Diseñar un sistema de catálogo e inventario con validación de stock en tiempo real y un carrito de compras persistente que gestione los productos seleccionados por el cliente.
- Desarrollar un sistema de gestión de pedidos y pagos con control de estados secuenciales, trazabilidad del ciclo de vida de cada compra y soporte para aprobaciones y reembolsos.
- Incorporar pruebas automatizadas y pipelines de CI/CD para garantizar la calidad, estabilidad y mantenibilidad del software entregado.

4. Planteamiento de la Solución

El equipo de desarrollo propone construir una WebApp con API-REST que reemplace completamente la plataforma actual. La solución incluirá gestión de clientes con autenticación segura, catálogo de productos con validación de stock en tiempo real, carrito de compras persistente entre sesiones, procesamiento de pedidos con seguimiento de estados, integración con pasarela de pago y notificaciones al cliente, aplicando los estándares de calidad ISO/IEC 25010 e IEEE 730.

Los criterios de éxito del proyecto contemplan reducir la tasa de abandono de carrito del 70% actual a menos del 30% en los primeros tres meses, y lograr un incremento de al menos el 20% en las ventas en línea durante el primer trimestre posterior a la implementación.

Se propone el desarrollo de una aplicación backend basada en FastAPI para gestionar las operaciones comerciales de Coquito Amarillo S.A.S. El sistema implementa una arquitectura modular orientada a servicios, permitiendo separar las responsabilidades relacionadas con autenticación, catálogo, pedidos, pagos e inventario.

A continuación, se describen los módulos que componen el sistema de forma detallada.

4.1. Módulo de Autenticación

El sistema incorpora un módulo de autenticación encargado de validar el acceso de los usuarios mediante credenciales seguras. Este componente controla las sesiones activas, restringe accesos no autorizados y protege las operaciones críticas del sistema.

4.2. Módulo de Catálogo

El módulo de catálogo permite administrar la información de los productos disponibles en la tienda virtual. Cada producto almacena información relacionada con nombre, descripción, categoría, precio, disponibilidad y stock. Asimismo, el sistema valida automáticamente la disponibilidad de inventario antes de permitir operaciones de compra.

4.3. Módulo de Carrito de Compras

El sistema incorpora un carrito de compras persistente que permite al cliente agregar, actualizar o eliminar productos antes de confirmar el pedido. Este módulo calcula subtotales, impuestos y totales generales de la compra.

4.4. Módulo de Pedidos

El módulo de pedidos administra el ciclo de vida completo de cada compra realizada por el cliente. Los pedidos pueden encontrarse en diferentes estados: pendiente, pagado, en preparación, enviado, entregado o cancelado. El sistema controla las transiciones permitidas entre estados con el fin de mantener la integridad del flujo operativo y garantizar la trazabilidad del proceso logístico.

4.5. Módulo de Pagos

El sistema implementa un módulo de pagos encargado de gestionar las transacciones asociadas a cada pedido. Este componente permite simular pagos electrónicos y validar el estado de las operaciones realizadas. Además, el sistema contempla procesos de reembolso en caso de cancelación de pedidos previamente aprobados.

4.6. Módulo de Notificaciones

El sistema genera notificaciones relacionadas con cambios de estado, confirmaciones de pago, cancelaciones y actualizaciones del pedido, proporcionando retroalimentación constante al usuario.

5. Actores del Sistema

Para el correcto modelado del sistema, se identificaron los siguientes actores que interactúan directamente en el entorno de la WebApp:

Actor	Tipo	Descripción
Cliente	Actor principal	Representa al usuario que interactúa con la plataforma realizando compras y gestionando pedidos.
Administrador	Actor secundario	Gestiona productos, inventario y operaciones administrativas del sistema.
Sistema de Pagos	Actor externo	Simula el procesamiento y validación de transacciones electrónicas.

6. Arquitectura del sistema

El sistema fue desarrollado utilizando una arquitectura en capas orientada a servicios REST. La capa de presentación está compuesta por el cliente HTTP y la interfaz Swagger UI. La capa de lógica de negocio está implementada con FastAPI organizada en módulos de servicio, encargados de aplicar las reglas de negocio, validaciones y transformaciones de datos. La capa de datos está gestionada por SQLAlchemy como ORM y SQLite como motor de base de datos relacional. La comunicación entre capas se realiza exclusivamente a través de la API-REST, garantizando una separación clara de responsabilidades.

Componente	Descripción
API REST	Expone los endpoints encargados de gestionar las operaciones del sistema.
Capa de Servicios	Implementa las reglas de negocio y validaciones principales.
Capa de Persistencia	Gestiona el acceso y almacenamiento de datos mediante SQLAlchemy.
Base de Datos	Almacena información relacionada con usuarios, productos, pedidos y pagos.

6.1. Módulos del Sistema

Módulo	Archivo Principal	Responsabilidad
Autenticación	routers/auth.py	Registro, login y cierre de sesión de clientes. Bloqueo por intentos fallidos.
Catálogo	routers/productos.py	CRUD de productos, búsqueda, filtrado por categoría y control de stock.
Carrito	routers/carrito.py	Gestión de ítems, persistencia en base de datos y recálculo del total.
Pedidos	routers/pedidos.py	Registro de pedidos, gestión de estados y generación de orden de despacho.
Pagos	routers/pagos.py	Procesamiento de pago, generación de comprobante y reembolso.
Notificaciones	services/notificaciones.py	Alertas internas en plataforma y envío de correos/SMS al cliente.

Modelos	models/	Definición de tablas con SQLAlchemy: Cliente, Producto, Carrito, Pedido, Pago.
Base de datos	database.py	Configuración de la conexión SQLite, sesiones y base declarativa.
Pruebas	tests/	Pruebas unitarias y de integración con pytest y httpx.

7. Requerimientos del sistema

Los requerimientos del sistema definen las condiciones y capacidades que debe cumplir la solución propuesta para garantizar su adecuado funcionamiento. Estos se derivan del análisis de los actores del negocio y de los problemas identificados en la plataforma actual. Los requerimientos se clasifican en requerimientos funcionales y no funcionales.

7.1. Requerimientos Funcionales

Los requerimientos funcionales describen las acciones y comportamientos que el sistema debe ejecutar:

ID	Nombre	Descripción
RF-001	Registro de cliente	El sistema permitirá que un visitante se registre con nombre, correo y contraseña única. El correo no puede estar previamente registrado.
RF-002	Inicio de sesión	El sistema autenticará clientes con correo y contraseña. Tras 5 intentos fallidos consecutivos, la cuenta quedará bloqueada temporalmente.
RF-003	Cierre de sesión	El sistema permitirá al cliente cerrar sesión de forma segura. El contenido del carrito se conservará en base de datos para la próxima sesión.
RF-004	Catálogo de productos	El sistema mostrará el catálogo de productos deportivos con funcionalidades de búsqueda por nombre y filtrado por categoría, precio y talla.
RF-005	Validación de stock en catálogo	El sistema consultará el stock en tiempo real. Los productos agotados se mostrarán con etiqueta "Agotado" y no podrán agregarse al carrito.
RF-006	Carrito persistente	El sistema almacenará el carrito del cliente en base de datos, conservando su contenido entre recargas de página y cierres de sesión.
RF-007	Gestión de ítems del carrito	El cliente podrá agregar productos al carrito, modificar la cantidad de cada ítem y eliminar productos. El total se recalculará automáticamente.
RF-008	Validación de stock en carrito	El sistema impedirá agregar más unidades de las disponibles en stock. Si el stock es insuficiente, mostrará una notificación.
RF-009	Recálculo automático del total	El sistema actualizará el precio total del carrito en menos de 1 segundo tras cualquier modificación de cantidad o eliminación de ítem.

RF-010	Resumen previo al pago	El sistema mostrará un resumen completo del pedido antes de procesar el pago. El cliente deberá confirmar explícitamente.
RF-011	Procesamiento de pago	El sistema integrará una pasarela de pago que acepte tarjeta, PSE, Nequi y Efecty. El resultado se reflejará en el estado del pedido.
RF-012	Retroalimentación del pago	El sistema mostrará al cliente el resultado del pago en menos de 3 segundos. En caso de error, indicará el motivo y ofrecerá la opción de reintentar.
RF-013	Confirmación de compra	El sistema enviará un correo electrónico al cliente con el resumen del pedido en menos de 1 minuto tras la aprobación del pago.
RF-014	Registro de pedido	El sistema registrará el pedido con un identificador único, asociado al cliente, los ítems, el total y la dirección de entrega, en estado "pendiente".
RF-015	Gestión de estados del pedido	El sistema gestionará los estados: pendiente → pagado → en_preparacion → enviado → entregado. El estado "cancelado" es alcanzable desde "pendiente" o "pagado".
RF-016	Cancelación manual de pedido	El cliente podrá cancelar un pedido en estado "pendiente" o "pagado". El sistema restaurará el stock y notificará al cliente.
RF-017	Cancelación automática por fallo	El sistema cancelará automáticamente el pedido si la pasarela rechaza la transacción, restaurará el stock y notificará al cliente.
RF-018	Historial de pedidos	El cliente podrá consultar el listado de sus pedidos ordenados por fecha, con detalle de productos, cantidades, total, estado y número de seguimiento.
RF-019	Generación de orden de despacho	El sistema generará automáticamente la orden de despacho al operador logístico al pasar el pedido a "en_preparacion". Reintentará hasta 3 veces ante fallos.
RF-020	Actualización de estados logísticos	El sistema recibirá actualizaciones del operador logístico vía API y reflejará el nuevo estado del pedido en la plataforma.
RF-021	Manejo de errores logísticos	El sistema gestionará fallos en la integración con el operador logístico, reintentará hasta 3 veces y notificará al cliente si el problema persiste.
RF-022	Notificaciones internas	El sistema mostrará alertas dentro de la plataforma para eventos relevantes: stock agotado, resultado del pago y cambios de estado del pedido.
RF-023	Notificaciones externas	El sistema enviará correo electrónico y SMS al cliente en: confirmación de compra, cambio de estado del pedido y cancelación.
RF-024	Gestión de inventario	El administrador podrá registrar, actualizar y desactivar productos. Cada producto tendrá nombre, descripción, precio, categoría, talla y stock disponible.
RF-025	Gestión de pedidos	El administrador podrá visualizar todos los pedidos, filtrarlos por estado y fecha, y actualizar manualmente el estado de un pedido cuando sea necesario.

7.2. Requerimientos No Funcionales

Los requerimientos no funcionales establecen las características de calidad, rendimiento y restricciones bajo las cuales el sistema debe operar.

ID	Nombre	Descripción	Prioridad	Métrica de Aceptación
RNF-001	Rendimiento	Las páginas cargarán en menos de 2 segundos bajo una conexión 4G simulada.	Media	Tiempo de carga < 2s medido con Lighthouse.
RNF-002	Disponibilidad	El sistema garantizará un uptime mínimo del 99.5% mensual.	Media	Uptime $\geq 99.5\%$ medido mensualmente.
RNF-003	Seguridad	Todas las comunicaciones usarán HTTPS. Las contraseñas se almacenarán con bcrypt con factor de costo mínimo de 12.	Alta	100% de endpoints con HTTPS y bcrypt cost ≥ 12 .
RNF-004	Usabilidad	Un usuario sin asistencia podrá completar el flujo de compra completo en menos de 5 minutos.	Media	Prueba con 10 usuarios reales; tiempo promedio < 5 minutos.
RNF-005	Escalabilidad	El sistema soportará picos de tráfico de hasta 500 usuarios concurrentes sin degradación del rendimiento.	Media	500 usuarios concurrentes; tiempo de respuesta < 3s y tasa de error < 1%.
RNF-006	Compatibilidad	El sistema funcionará correctamente en Chrome, Firefox y Safari en sus dos últimas versiones, en escritorio y móvil.	Media	Pruebas en Chrome, Firefox y Safari en resoluciones 1280px y 375px.
RNF-007	Tiempo de respuesta pasarela	El sistema esperará un máximo de 15 segundos la respuesta de la pasarela. Si se supera, mostrará error y cancelará la transacción.	Alta	Timeout configurado a 15 s; verificado en pruebas de integración.
RNF-008	Privacidad de datos	El sistema cumplirá con la Ley 1581 de 2012. Los datos de pago nunca se almacenarán en texto plano.	Alta	Auditoría de cumplimiento de la Ley 1581; datos de pago cifrados.

8. Casos de Uso

La siguiente tabla consolida los doce casos de uso del sistema FitXploit, validados contra los requerimientos funcionales.

ID	Nombre	Actor	RF
CU-01	Registrarse e iniciar sesión	Cliente	RF-001, RF-002, RF-003
CU-02	Explorar catálogo de productos	Cliente	RF-004, RF-005
CU-03	Gestionar carrito de compras	Cliente	RF-006, RF-007, RF-008, RF-009
CU-04	Registrar pedido	Cliente	RF-010, RF-014
CU-05	Procesar pago	Cliente	RF-011, RF-012, RF-013
CU-06	Cancelar pedido	Cliente	RF-016, RF-017
CU-07	Consultar historial de pedidos	Cliente	RF-018
CU-08	Gestionar inventario de productos	Administrador	RF-024
CU-09	Gestionar pedidos del sistema	Administrador	RF-015, RF-025
CU-10	Actualizar estado logístico del pedido	Operador Logístico	RF-019, RF-020, RF-021

A continuación se presenta la especificación completa de cada caso de uso CU

8.1. CU-01: Registrarse e iniciar sesión

Identificador	CU-01
Nombre	Registrarse e iniciar sesión
Actor principal	Cliente
RF que cubre	RF-001, RF-002, RF-003
Precondición	El sistema se encuentra en ejecución. Para login: el cliente ya tiene cuenta registrada.
Flujo principal	1. El visitante ingresa nombre, correo y contraseña para registrarse. 2. El sistema valida que el correo no esté previamente registrado. 3. El sistema almacena la contraseña cifrada con bcrypt y devuelve el id_cliente. 4. El cliente ingresa su correo y contraseña para iniciar sesión. 5. El sistema valida las credenciales y confirma el acceso con el nombre del cliente.
Flujo alterno	Si el correo ya existe, el sistema rechaza el registro con un mensaje de error. Si las credenciales son incorrectas, el sistema registra el intento fallido. Tras 5 intentos fallidos consecutivos, la cuenta queda bloqueada temporalmente (RF-002). Si el correo no existe, el sistema muestra un mensaje de error claro.
Postcondición	El cliente queda registrado en el sistema o accede a su sesión activa.

8.2. CU-02: Explorar catálogo de productos

Identificador	CU-02
Nombre	Explorar catálogo de productos
Actor principal	Cliente
RF que cubre	RF-004, RF-005
Precondición	El cliente ha iniciado sesión correctamente.
Flujo principal	1. El cliente accede al módulo de catálogo. 2. El sistema muestra únicamente los productos activos con nombre, precio, stock y categoría. 3. El cliente aplica filtros por categoría, precio o busca por nombre. 4. El sistema consulta el stock en tiempo real y etiqueta como Agotado los productos sin disponibilidad. 5. El cliente selecciona un producto para ver su detalle.
Flujo alternativo	Si el producto solicitado no existe, el sistema muestra un mensaje de error. Si un producto está agotado, se muestra la etiqueta Agotado y se deshabilita la opción de agregarlo al carrito.
Postcondición	El cliente visualiza el catálogo actualizado con disponibilidad de stock en tiempo real.

8.3. CU-03: Gestionar carrito de compras

Identificador	CU-03
Nombre	Gestionar carrito de compras
Actor principal	Cliente
RF que cubre	RF-006, RF-007, RF-008, RF-009
Precondición	El cliente ha iniciado sesión y existe al menos un producto activo con stock disponible.
Flujo principal	1. El cliente agrega un producto al carrito. 2. El sistema valida que haya stock suficiente; si el producto ya estaba en el carrito, suma la cantidad. 3. El cliente modifica la cantidad de un ítem o lo elimina. 4. El sistema valida que la nueva cantidad no supere el stock disponible. 5. El sistema recalcula subtotales y total general en menos de 1 segundo. 6. El carrito se persiste en base de datos y se conserva entre sesiones y cierres de sesión.
Flujo alternativo	Si el stock es insuficiente al agregar o modificar, el sistema muestra una notificación y no permite la operación. Si el cliente elimina todos los ítems, el carrito queda vacío sin errores.
Postcondición	El carrito del cliente queda actualizado y persistido en base de datos con los productos y cantidades seleccionados.

8.4. CU-04: Registrar pedido

Identificador	CU-04
Nombre	Registrar pedido
Actor principal	Cliente
RF que cubre	RF-010, RF-014
Precondición	El cliente ha iniciado sesión y tiene al menos un producto en el carrito.
Flujo principal	1. El cliente confirma el contenido del carrito y proporciona la dirección de entrega. 2. El sistema muestra el resumen completo del pedido con ítems, cantidades y total. 3. El cliente confirma explícitamente el pedido. 4. El sistema valida el stock de cada producto antes de confirmar. 5. El sistema registra el pedido con un identificador único en estado pendiente y vacía el carrito automáticamente.
Flujo alternativo	Si el carrito está vacío, el sistema impide registrar el pedido. Si algún producto no tiene stock suficiente al momento de confirmar, el sistema notifica al cliente y no genera el pedido.
Postcondición	El pedido queda registrado en el sistema con un ID único, asociado al cliente, los ítems, el total y la dirección de entrega, en estado pendiente.

8.5. CU-05: Procesar pago

Identificador	CU-05
Nombre	Procesar pago
Actor principal	Cliente
RF que cubre	RF-011, RF-012, RF-013
Precondición	Existe un pedido registrado en estado pendiente asociado al cliente.
Flujo principal	1. El cliente selecciona el método de pago (tarjeta, PSE, Nequi o Efecty). 2. El sistema integra la pasarela de pago y procesa la transacción. 3. El sistema muestra el resultado del pago en menos de 3 segundos. 4. Si el pago es aprobado, el sistema actualiza el pedido a estado pagado, genera una referencia de transacción y envía confirmación por correo electrónico al cliente en menos de 1 minuto.

Flujo alternativo	Si la pasarela rechaza el pago, el sistema indica el motivo del error y ofrece la opción de reintentar. Si se supera el timeout de 15 segundos, el sistema cancela la transacción, cancela el pedido automáticamente, restaura el stock y notifica al cliente (RF-017).
Postcondición	El pedido queda en estado pagado con comprobante generado, o en estado cancelado si el pago falló.

8.6. CU-06: Cancelar pedido

Identificador	CU-06
Nombre	Cancelar pedido
Actor principal	Cliente
RF que cubre	RF-016, RF-017
Precondición	Existe un pedido asociado al cliente en estado pendiente o pagado.
Flujo principal	1. El cliente solicita la cancelación de un pedido. 2. El sistema verifica que el pedido se encuentre en un estado cancelable (pendiente o pagado). 3. El sistema cancela el pedido y restaura el stock de todos los productos asociados. 4. El sistema notifica al cliente sobre la cancelación.
Flujo alternativo	Si el pedido ya se encuentra en estado en_preparacion, enviado o entregado, el sistema rechaza la cancelación e informa al cliente que no es posible.
Postcondición	El pedido queda en estado cancelado y el inventario de los productos asociados es restablecido automáticamente.

8.7. CU-07: Consultar historial de pedidos

Identificador	CU-07
Nombre	Consultar historial de pedidos
Actor principal	Cliente
RF que cubre	RF-018
Precondición	El cliente ha iniciado sesión correctamente.
Flujo principal	1. El cliente accede al módulo de historial de pedidos. 2. El sistema recupera y muestra todos los pedidos del cliente ordenados por fecha. 3. Para cada pedido se muestra el ID, productos, cantidades, total, estado y número de seguimiento.

Flujo alternativo	Si el cliente no tiene pedidos registrados, el sistema muestra un mensaje informativo indicando que no hay compras previas.
Postcondición	El cliente visualiza el historial completo de sus pedidos con todos los detalles disponibles.

8.8. CU-08: Gestionar inventario de productos

Identificador	CU-08
Nombre	Gestionar inventario de productos
Actor principal	Administrador
RF que cubre	RF-024
Precondición	El administrador tiene acceso al sistema.
Flujo principal	1. El administrador registra un nuevo producto con nombre, descripción, precio, categoría, talla y stock. 2. El sistema valida que no exista un producto duplicado con el mismo nombre y categoría. 3. El producto queda activo y visible en el catálogo inmediatamente. 4. El administrador puede actualizar precio o stock de un producto existente; el sistema valida que el stock no sea negativo. 5. El administrador puede desactivar un producto para ocultarlo del catálogo sin eliminarlo permanentemente.
Flujo alternativo	Si se intenta registrar un producto duplicado, el sistema lo rechaza con un mensaje de error. Si el stock ingresado es negativo, el sistema no permite la operación. Un producto desactivado puede reactivarse en cualquier momento.
Postcondición	El catálogo queda actualizado y los cambios se reflejan en tiempo real para los clientes.

8.9. CU-09: Gestionar pedidos del sistema

Identificador	CU-09
Nombre	Gestionar pedidos del sistema
Actor principal	Administrador
RF que cubre	RF-015, RF-025
Precondición	El administrador tiene acceso al sistema y existen pedidos registrados.

Flujo principal	1. El administrador accede al módulo de gestión de pedidos. 2. El sistema muestra todos los pedidos, con opción de filtrar por estado y fecha. 3. El administrador selecciona un pedido y actualiza su estado manualmente cuando sea necesario. 4. El sistema valida que la transición de estado sea permitida según el flujo definido (pendiente → pagado → en_preparacion → enviado → entregado). 5. El sistema notifica al cliente sobre el cambio de estado.
Flujo alterno	Si se intenta una transición de estado no permitida, el sistema la rechaza e informa al administrador.
Postcondición	El pedido queda actualizado con el nuevo estado y el cliente recibe la notificación correspondiente.

8.10. CU-10: Actualizar estado logístico del pedido

Identificador	CU-10
Nombre	Actualizar estado logístico del pedido
Actor principal	Operador Logístico
RF que cubre	RF-019, RF-020, RF-021
Precondición	El pedido se encuentra en estado pagado o posterior, y el operador logístico tiene acceso a la API.
Flujo principal	1. Al pasar un pedido a en_preparacion, el sistema genera automáticamente la orden de despacho y la envía al operador logístico. 2. El operador logístico envía una actualización de estado vía API. 3. El sistema recibe la actualización y refleja el nuevo estado del pedido en la plataforma. 4. El sistema notifica al cliente sobre el cambio de estado logístico.
Flujo alterno	Si la generación de la orden de despacho falla, el sistema reintenta hasta 3 veces antes de notificar el error. Si falla la integración con el operador logístico al recibir actualizaciones, el sistema reintenta hasta 3 veces y notifica al cliente si el problema persiste (RF-021).
Postcondición	El estado logístico del pedido queda actualizado en la plataforma y el cliente recibe la notificación correspondiente.

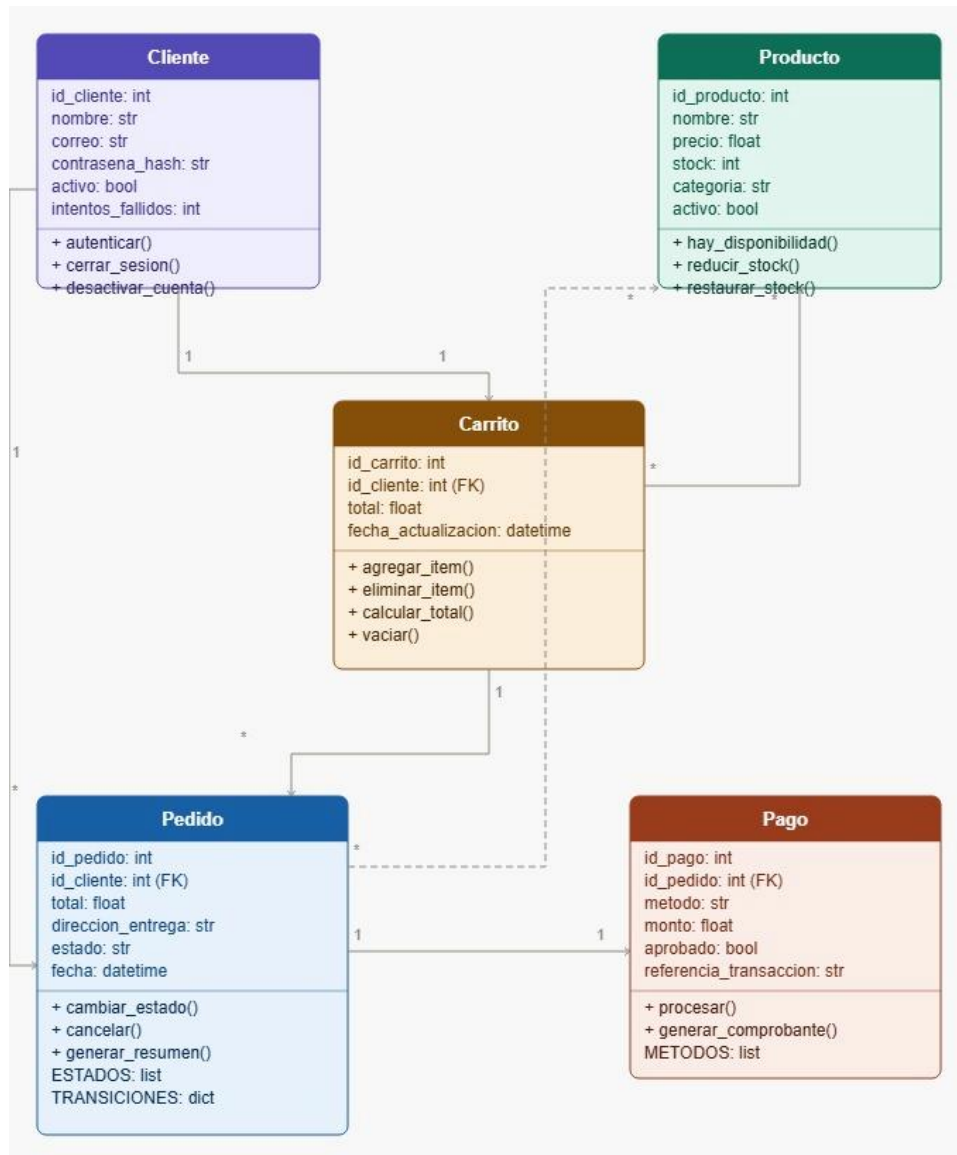
9. Clases del Sistema

Clase	Atributos	Métodos
Cliente	id_cliente, nombre, correo, contraseña_hash, activo, intentos_fallidos, fecha_registro	autenticar(), cerrar_sesion(), desactivar_cuenta()
Producto	id_producto, nombre, precio, stock, categoria, activo	hay_disponibilidad(), reducir_stock(), restaurar_stock()
Carrito	id_carrito, id_cliente (FK), total, fecha_actualizacion	agregar_item(), modificar_cantidad(), eliminar_item(), calcular_total(), vaciar()
ItemCarrito	id_item, id_carrito (FK), id_producto (FK), cantidad, subtotal	—
Pedido	id_pedido, id_cliente (FK), total, direccion_entrega, estado, fecha, numero_seguimiento	cambiar_estado(), cancelar(), generar_resumen()
ItemPedido	id_item, id_pedido (FK), id_producto (FK), cantidad, subtotal	—
Pago	id_pago, id_pedido (FK), metodo, monto, aprobado, fecha_pago, referencia_transaccion	procesar(), generar_comprobante()

9.1.Relaciones Entre Clases

Clase Origen	Multiplicidad	Clase Destino	Tipo
Cliente	1 — 1	Carrito	Composición
Cliente	1 — *	Pedido	Asociación
Carrito	1 — *	ItemCarrito	Composición
Pedido	1 — *	ItemPedido	Composición
Pedido	1 — 1	Pago	Composición
ItemCarrito	* — 1	Producto	Asociación
ItemPedido	* — 1	Producto	Asociación

10. Diagramas de Clase



11. Organización del Equipo

11.1. Integrantes y Roles

Integrante	Rol Principal	Responsabilidades
Johan Sebastián Barrera Rua	Backend Developer	API-REST, modelos de datos, pruebas unitarias, CI/CD con GitHub Actions.
Miguel Ángel González Estrada	Full Stack Developer	Frontend, integración API, documentación técnica, despliegue en Codespaces.

11.2. Metodología de Trabajo

El equipo adopta la metodología Scrum con entregas semanales y retrospectivas, conforme a lo establecido en la guía del curso. Las actividades se gestionan mediante GitHub Issues y el repositorio oficial del proyecto, disponible en https://github.com/sebasbarrera14/Practica_Final_SB_MG

El control de versiones se realiza con Git, organizando el trabajo en ramas feature/ para nuevas funcionalidades, fix/ para correcciones y main para la rama de producción estable. Todo código nuevo es revisado mediante Pull Requests con aprobación mutua obligatoria antes de hacer merge a main.

12. Normas y Procedimientos de Calidad

12.1. ISO/IEC 25010 — Modelo de Calidad del Producto

Característica	Descripción	Métrica de Aceptación
Funcionalidad	El sistema cumple todos los requisitos funcionales definidos.	100% de RF cubiertos por pruebas de aceptación.
Rendimiento	Tiempo de respuesta adecuado bajo carga normal.	Tiempo de carga < 2 segundos con conexión 4G simulada.
Usabilidad	El usuario completa una compra sin asistencia externa.	Flujo completo en menos de 5 minutos.
Fiabilidad	El sistema opera sin interrupciones no planificadas.	Uptime $\geq 99.5\%$ mensual.
Seguridad	Protección de datos personales y accesos no autorizados.	HTTPS en todos los endpoints; bcrypt cost ≥ 12 .
Mantenibilidad	Código limpio, modular y documentado para facilitar cambios futuros.	pylint ≥ 8.0 ; complejidad ciclomática ≤ 10 por función.

12.2. IEEE 730 — Plan de Aseguramiento de Calidad

Conforme al estándar IEEE 730, el equipo aplica las siguientes actividades de aseguramiento de calidad a lo largo del ciclo de desarrollo: revisiones de código mediante Pull Requests antes de cada merge a main; ejecución automática de pruebas unitarias en cada push mediante GitHub Actions; análisis estático de código con pylint con score mínimo de 8.0; pruebas de integración sobre los endpoints de la API con Postman y pytest; y registro y seguimiento de defectos mediante GitHub Issues.

13. Actividades de Aseguramiento de Calidad

13.1. Estrategia de Pruebas

El equipo aplica la metodología TDD (Test Driven Development), escribiendo las pruebas antes de implementar cada función. La estrategia de pruebas se organiza en tres niveles complementarios.

Nivel	Herramienta	Cobertura Esperada
Pruebas Unitarias	pytest	Cada función de negocio con casos típicos, de límite y excepcionales.
Pruebas de Integración	pytest + httpx	Todos los endpoints de la API-REST (happy path y flujos alternativos).
Pruebas de Aceptación	Postman	Escenarios derivados de los 13 casos de uso documentados.

13.2. Criterios de Aceptación por Módulo

Cada módulo del sistema tiene criterios de aceptación específicos que deben cumplirse antes de considerarlo completado: el módulo de autenticación debe soportar registro, login y bloqueo tras 5 intentos fallidos; el catálogo debe permitir búsqueda, filtrado y validación de stock en tiempo real; el carrito debe mantener persistencia entre sesiones, recalcular el total ante cada cambio y validar el stock disponible; el módulo de pedidos debe registrar con ID único, gestionar los estados definidos y generar la orden de despacho; el módulo de pagos debe procesar con al menos 4 métodos, generar comprobante por correo y soportar reembolso; y el módulo de notificaciones debe enviar alertas internas y correos/SMS en todos los eventos clave del proceso de compra.

14. Herramientas y Tecnologías

La selección del Stack tecnológico se realizó considerando la continuidad con el trabajo previo del curso, la productividad del equipo y los requerimientos de rendimiento y escalabilidad del proyecto.

Nivel	Herramienta	Cobertura Esperada
Lenguaje	Python 3.11	Continuidad con el trabajo previo del curso; amplio ecosistema de librerías.
Framework Backend	FastAPI	Alto rendimiento, documentación automática con Swagger UI, tipado fuerte con Pydantic.
Base de Datos	SQLite	Sin configuración de servidor, ideal para el alcance del proyecto y entornos de desarrollo.
ORM	SQLAlchemy	Abstracción de la base de datos, evita SQL crudo y facilita migraciones.
Pruebas	pytest + httpx	Compatibles con FastAPI; permiten TDD de forma simple y pruebas de integración sobre la API.
Control de Versiones	Git + GitHub	Repositorio: https://github.com/sebasbarrera14/Practica_Final_SB_MG
Entorno de Desarrollo	GitHub Codespaces	Requerido por el plan de clase para desarrollo y despliegue en la nube.
CI/CD	GitHub Actions	Ejecución automática de pruebas en cada push a main, garantizando integración continua.
Documentación API	Swagger UI (integrado)	FastAPI genera automáticamente la documentación interactiva de todos los endpoints.
Testing API	Postman	Pruebas de integración manuales y validación de escenarios de aceptación.

15. Métricas y Criterios de Aceptación

15.1. Métricas de Calidad del Código

Métrica	Valor Mínimo Aceptable	Herramienta de Medición
Score de calidad de código	8.0 / 10.0	pylint
Complejidad ciclomática	≤ 10 por función	radon cc
Cobertura de pruebas unitarias	$\geq 80\%$	pytest-cov
Cobertura de RF por pruebas de aceptación	$\geq 90\%$	Matriz de trazabilidad
Tiempo de respuesta de la API	< 500 ms por endpoint	Postman / httpx
Tiempo de carga de la WebApp	< 2 segundos	Lighthouse

15.2. Criterios de Aceptación del Producto Final

El producto se considerará aceptado cuando se cumplan todos los siguientes criterios: todos los endpoints de la API-REST responden correctamente con los códigos HTTP esperados; las pruebas unitarias y de integración pasan al 100% en el pipeline de CI/CD; la documentación Swagger UI está disponible y actualizada en el entorno de despliegue; el flujo completo de compra (registro $\rightarrow$ catálogo $\rightarrow$ carrito $\rightarrow$ pago $\rightarrow$ notificación) funciona de extremo a extremo; y el sistema gestiona correctamente los estados del pedido según las transiciones definidas.

16. Implementación del Sistema

16.1. Backend y API REST

El sistema fue desarrollado utilizando FastAPI como framework principal para la construcción de servicios REST. Esta tecnología permitió implementar endpoints organizados, documentación automática y validaciones mediante esquemas definidos con Pydantic.

16.2. Base de Datos

La persistencia de la información se implementó utilizando SQLAlchemy y SQLite. El sistema almacena información relacionada con clientes, productos, pedidos, pagos y movimientos de inventario.

16.3. Gestión de Pedidos

El módulo de pedidos controla el ciclo de vida de las compras realizadas por los clientes. El sistema valida las transiciones permitidas entre estados y garantiza la consistencia del inventario durante cada operación. Cuando un pedido es cancelado antes del despacho, el sistema restablece automáticamente el stock de los productos asociados.

16.4. Gestión de Pagos

El sistema incorpora un módulo encargado de simular el procesamiento de pagos electrónicos. Cada transacción almacena información relacionada con monto, método de pago, referencia y estado de aprobación. Asimismo, el sistema permite iniciar procesos de reembolso cuando un pedido previamente aprobado es cancelado.

16.5. Validaciones y Seguridad

La aplicación implementa validaciones relacionadas con autenticación, integridad de datos, disponibilidad de inventario y restricciones sobre operaciones críticas. Las reglas de negocio fueron centralizadas en servicios especializados para garantizar coherencia y reutilización del código.

16.6. Pruebas y CI/CD

El proyecto implementa pruebas automatizadas utilizando pytest con el fin de validar el correcto funcionamiento de los módulos principales. Asimismo, se incorporaron procesos de integración continua mediante GitHub Actions para ejecutar pruebas automáticas y verificar la estabilidad del sistema antes de cada integración.

17. Gestión de Cambios

17.1. Procedimiento de Control de Cambios

Todo cambio al alcance, requisitos o arquitectura del sistema debe seguir el siguiente procedimiento antes de ser implementado: el integrante que identifica el cambio abre un Issue en GitHub describiendo qué cambia, por qué cambia y qué impacto tiene sobre otros módulos; el equipo revisa el Issue en la siguiente reunión de sincronización y evalúa el impacto en tiempo, pruebas y documentación; si el cambio es aprobado, se asigna a una rama feature/nombre-cambio y se implementa con sus respectivas pruebas; se realiza un Pull Request a main con revisión obligatoria del otro integrante antes de hacer merge; y finalmente se actualiza la documentación técnica y el README del repositorio para reflejar el cambio.

17.2. Clasificación de Cambios

Tipo	Descripción	Aprobación Requerida
Menor	Corrección de bugs, mejoras de estilo, ajustes de documentación.	Un integrante puede aprobar el Pull Request.
Mayor	Nuevos endpoints, cambios en modelos de datos, nuevas dependencias.	Ambos integrantes deben aprobar el Pull Request.
Crítico	Cambios en la arquitectura, eliminación de módulos, cambio de tecnología.	Aprobación del docente más ambos integrantes.

18. Operación, Pruebas y Mejoras

El sistema puede ejecutarse localmente mediante FastAPI y accederse a través de la documentación Swagger generada automáticamente.

Las pruebas realizadas permitieron validar: registro y autenticación de usuarios; gestión del catálogo de productos; operaciones del carrito de compras; procesamiento de pedidos; validación de pagos; actualización de estados; y restauración de inventario.

Como mejoras futuras se contemplan: integración con pasarelas de pago reales; implementación de autenticación JWT; incorporación de notificaciones por correo electrónico; migración a bases de datos empresariales; despliegue en servicios cloud; e implementación de microservicios.

19. Revisión y Aprobación

19.1. Cronograma de Sprints

Sprint	Entregable	Criterio de Completitud
Sprint 1	Estructura del repositorio, modelos de datos y endpoints de autenticación.	Pruebas de registro y login pasando al 100%.
Sprint 2	Módulos de catálogo y carrito con validación de stock.	Pruebas de integración de carrito completadas.
Sprint 3	Módulos de pedidos, pagos y notificaciones.	Flujo completo de compra funcionando de extremo a extremo.

19.2. Firmas de Aprobación

Rol	Nombre	Firma	Fecha
Desarrollador	Johan Sebastián Barrera Rua		
Desarrollador	Miguel Ángel González Estrada		
Docente Supervisor	Luis Fernando Sánchez		

20. Reflexión

El desarrollo del sistema permitió aplicar conceptos fundamentales de ingeniería de software relacionados con arquitectura, modelado, diseño modular y control de calidad.

La definición de requerimientos funcionales y no funcionales facilitó la construcción de una solución coherente con las necesidades de la organización, mientras que el modelado UML permitió anticipar el comportamiento del sistema antes de la implementación.

Asimismo, la separación de responsabilidades mediante una arquitectura modular favoreció la mantenibilidad, reutilización y escalabilidad del software desarrollado. La implementación de pruebas automatizadas y procesos de integración continua evidenció la importancia de las prácticas modernas de aseguramiento de calidad dentro del ciclo de vida del software.

21. Conclusiones

El sistema desarrollado para Coquito Amarillo S.A.S. permitió construir una solución tecnológica orientada a optimizar los procesos de ventas, pedidos y gestión de inventario de la organización.

La implementación de una arquitectura basada en servicios REST facilitó la separación de responsabilidades y mejoró la mantenibilidad del proyecto. Además, la incorporación de mecanismos de validación, control de estados y pruebas automatizadas contribuyó a garantizar la confiabilidad y consistencia de las operaciones realizadas por el sistema.

Finalmente, el proyecto permitió consolidar conocimientos relacionados con desarrollo backend, modelado UML, bases de datos, control de calidad y arquitectura de software, fortaleciendo las competencias adquiridas durante el proceso de formación académica.